3.6.2023

Skript

Hadoop und MapReduce

Tobias Walter, Mtrk. Nr. 5095272
TECHNISCHE HOCHSCHULE MITTELHESSEN

Inhaltsverzeichnis

1. Einführung 1
1.1. Vorstellung des Themas 1
1.1.1. Hadoop 1
1.1.2. MapReduce 2
2. Hadoop-Ökosystem 2
2.1. Hadoop und seine Anwendungsbereiche 2
2.2 Hadoop-Architektur und Komponenten 4
2.3.1. HDFS 4
2.3.2. YARN 11
3. MapReduce-Programmiermodel 13
3.1. MapReduce Algorithmus 13
3.1. Apache MapReduce Programmiermodel 14
3.1.1. Map Task 15
3.1.2. Reduce Task 15
3.2.3 Java MapReduce Codebeispiel 15
4. Hadoop und MapReduce in der Praxis 18
4.1. Anwendungsbeispiele 18
4.1.1 Netflix 18
4.2.2 Facebook 18
4.2.3 Twitter 19
4.2. Limitationen von Hadoop MapReduce 19
5. Übung: MapReduce-Aufgabe 20
5.1. Erläuterung der Aufgabenstellung 20
5.2. Lösung zur Aufgabenstellung 21
I. Literatur 22

1. Einführung

1.1. Vorstellung des Themas

Dieses Skript beinhaltet eine Einführung in Hadoop und Apache Hadoop welches auf dem MapReduce-Programmiermodel aufbaut. Das Skript dient im Rahmen der Veranstaltung Datenbanken und Informationssysteme an der Technischen Hochschule Mittelhessen.

Das Skript bietet einen Überblick über den Aufbau und die Funktionsweise von Hadoop im Zusammenhang mit MapReduce, sowie weiteren Teilen des Hadoop-Ökosystems. Wir betrachten dabei die Thematik von Hadoop und MapReduce im Einzelnen und wie das Apache Hadoop Framework auf MapReduce aufgebaut ist.

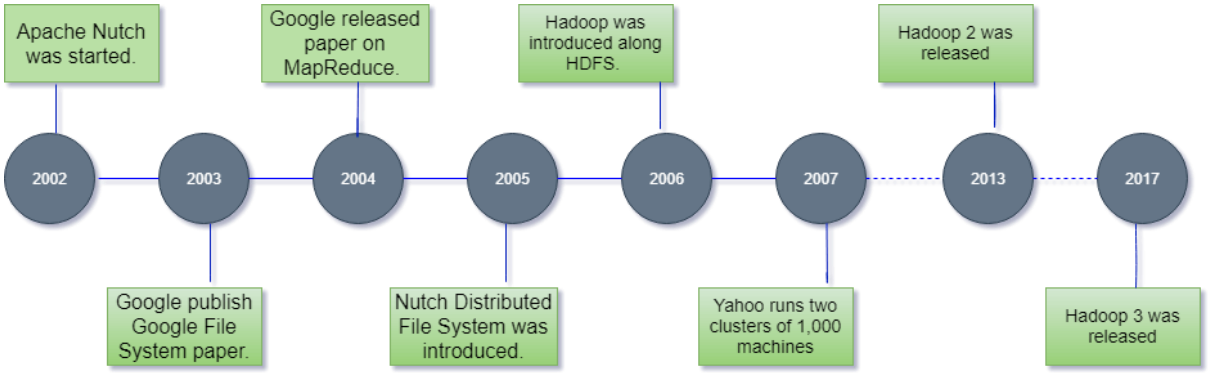


Abbildung 1 - Historie von Hadoop[1]

1.1.1. Hadoop

Hadoop ist ein System, das einen verteilten Dateisystemrahmen und eine Methode zur Analyse und Transformation von sehr großen Datensätzen über das MapReduce-Paradigma bereitstellt. Eine wesentliche Eigenschaft von Hadoop besteht darin, Daten und Berechnungen auf viele (tausende) Hosts zu verteilen und Anwendungsberechnungen parallel zu ihren Daten auszuführen.[2]

Das Hadoop System kann über die Apache Hadoop Softwarebibliothek, die auf der Programmiersprache Java basiert implementiert werden. Apache Hadoop ist ein Framework, welches die verteilte Verarbeitung umfangreicher Datensätze (Big Data) mittels einfacher Programmieransätze auf Clustern von Computern ermöglicht. Es wurde entwickelt, um von einzelnen Servern auf mehrere tausend Maschinen zu erweitern, wobei jede Maschine lokale Rechenkapazität und Speicherplatz bereitstellt.

[1] https://static.javatpoint.com/hadooppages/images/history-of-hadoop.png
[2] Shvachko u. a., 2010, S. 1

Anstatt sich auf die Hardware zu verlassen, um Hochverfügbarkeit zu bieten, ist die Bibliothek so konzipiert, dass sie Fehler auf der Anwendungsebene erkennt und behebt. Somit liefert sie einen hochverfügbaren Dienst auf einem Verbund von Computern, obwohl jeder dieser Computer anfällig für Ausfälle sein kann.[3]

1.1.2. MapReduce

MapReduce ist eine von Google entwickeltes Konzept eines verteilten Programmiermodells, welches als Reaktion auf die Herausforderungen bei der Verarbeitung großer Datenmengen entstanden ist. Die Konzeption ermöglicht das einfache Verarbeiten dieser Daten, während sie die komplexen Aspekte der Parallelisierung, Fehlertoleranz, Datenverteilung und Lastausgleich verbirgt. Inspiriert durch die Primitiven "Map" und "Reduce" funktionaler Programmiersprachen, ermöglicht MapReduce das Anwenden von Operationen auf jeden Eingabedatensatz, um Zwischenpaare von Schlüssel-Wert zu generieren (Map-Operation), und kombiniert dann alle Werte, die den gleichen Schlüssel teilen, um abgeleitete Daten zu erzeugen (Reduce-Operation).

Dieses Modell erleichtert die Parallelisierung von großen Berechnungen und nutzt Wiederholung als primären Mechanismus zur Fehlertoleranz. Die leistungsfähige Schnittstelle ermöglicht die automatische Parallelisierung und Verteilung großer Berechnungen und erreicht hohe Leistung auf Clustern von Standard-PCs.[4]

2. Hadoop-Ökosystem

2.1. Hadoop und seine Anwendungsbereiche

Wir leben im Zeitalter der Daten. Laut einer Schätzung von IDC betrug die Größe des "digitalen Universums" im Jahr 2013 4,4 Zettabytes und wird bis 2020 auf 44 Zettabytes ansteigen. Ein Zettabyte entspricht 1021 Bytes. Diese Datenmenge ist enorm und wächst weiter. Das Zeitalter von Big Data bringt Herausforderungen mit sich wenn auf einer so großen Menge von Daten effizient und zuverlässig gearbeitet werden soll:[5]

1. **Speicherung großer Datenmengen:** Die Speicherkapazitäten von Festplatten sind im Laufe der Jahre zwar stark gestiegen, aber die enormen Datenmengen stellen immer noch eine Herausforderung für die Speicherung dar.[6]

2. **Zugriffsgeschwindigkeit:** Trotz steigender Speicherkapazitäten haben die Zugriffsgeschwindigkeiten, also die Geschwindigkeit, mit der Daten von den Festplatten gelesen werden können, nicht im gleichen Maße zugenommen.[7]

---

[3] Apache, 2023, https://hadoop.apache.org/
[4] Jeffrey Dean and Sanjay Ghemawat, 2004, S.1
[5] White, 2015, S. 3-4
[6] White, 2015, S. 5
[7] White, 2015, S. 5

3. **Parallelisierung der Datenspeicherung und -zugriffe:** Eine Lösung für das Zugriffsgeschwindigkeitsproblem könnte das gleichzeitige Lesen von Daten von mehreren Festplatten sein, was eine effiziente Nutzung der Speicherhardware erfordert.[8]

4. **Ausfallrisiko bei Hardware:** Mit dem Einsatz vieler Hardwarekomponenten steigt das Risiko von Ausfällen. Diese Ausfälle können zum Verlust von Daten führen, was eine erhebliche Herausforderung darstellt.[9]

5. **Datenreplikation zur Vermeidung von Datenverlusten:** Eine gängige Methode zur Vermeidung von Datenverlust besteht darin, redundante Kopien der Daten im System zu speichern, sodass bei einem Ausfall eine Ersatzkopie zur Verfügung steht.[10]

6. **Kombination von Daten für die Analyse:** Die meisten Analyseaufgaben erfordern, dass Daten auf irgendeine Weise kombiniert werden. Dies ist eine Herausforderung, insbesondere wenn Daten auf mehreren Festplatten oder sogar auf verschiedenen Systemen verteilt gespeichert sind.[11]

Hier tritt Hadoop auf den Plan, um diese Probleme zu lösen. Zusammenfassend bietet Hadoop eine robuste und skalierbare Plattform für die Speicherung und Analyse großer Datenmengen, was es zu einem wichtigen Werkzeug in verschiedensten Anwendungsbereichen von Big Data, wie Business-Intelligence bis hin zu wissenschaftlicher Forschung, macht[12] :

1. **Skalierbare Speicherung:** Trotz ständig steigender Speicherkapazitäten bleiben die Zugriffsgeschwindigkeiten auf Festplatten hinterher. Hadoop adressiert dies durch verteilte Speicherung mithilfe des HDFS, was bedeutet, dass Daten gleichzeitig von mehreren Festplatten gelesen werden können.[13]

2. **Zuverlässigkeit durch Replikation:** Bei der Verwendung von zahlreichen Hardwarekomponenten steigt das Risiko von Ausfällen. Hadoop minimiert das Risiko von Datenverlusten durch redundante Speicherung der Daten über das HDFS. Bei einem Hardwareausfall stellt das System eine replizierte Kopie zur Verfügung.[14]

3. **Datenanalyse:** Die meisten Analyseaufgaben erfordern eine Kombination von Daten. Mit Hadoop's MapReduce wird die Analyse durch die Abstraktion des Problems in Schlüssel-Wert-Paare vereinfacht[15]

---

[8] White, 2015, S. 5
[9] White, 2015, S. 5
[10] White, 2015, S. 6
[11] White, 2015, S. 6
[12] White, 2015, S. 6
[13] White, 2015, S. 43-44
[14] White, 2015, S. 43-44
[15] White, 2015, S. 6

2.2 Hadoop-Architektur und Komponenten

Neben HDFS und MapReduce bietet das Hadoop-Ökosystem eine weitaus umfangreichere Palette an Komponenten und Tools für Datenmanagement und Verwaltung. In diesem Skript werden wir unseren Fokus dabei auf HDFS, MapReduce und YARN legen.

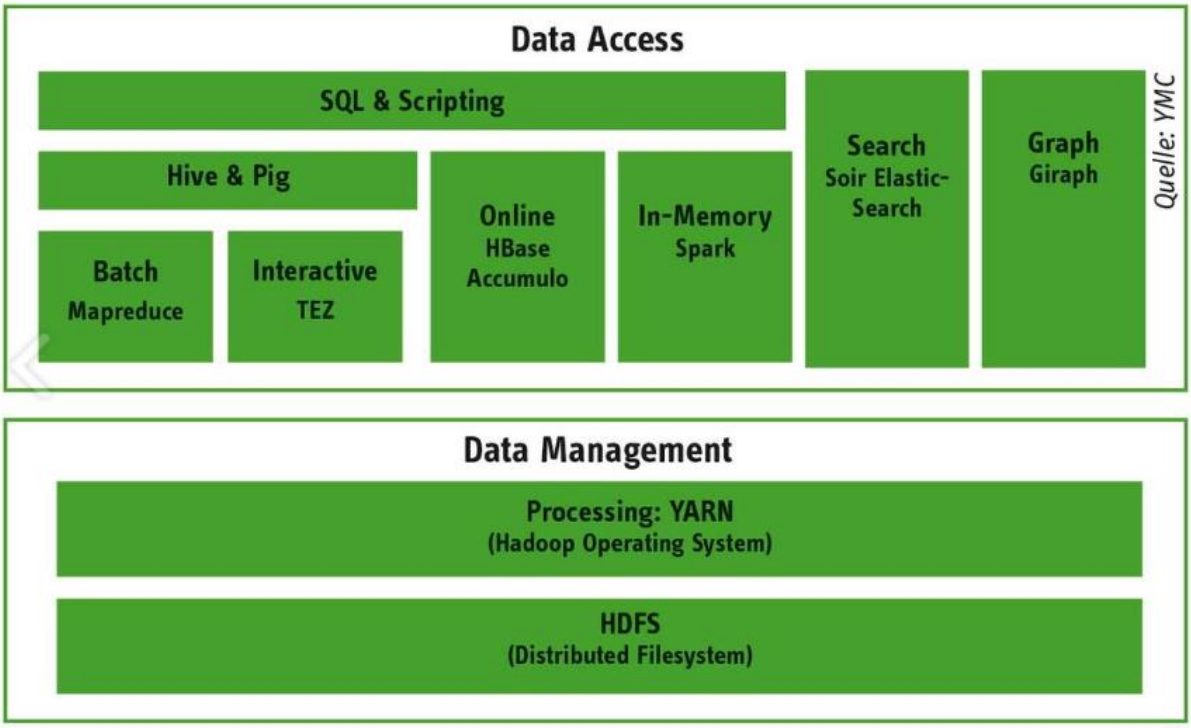


Abbildung 2 - Hadoop Ökosystem[16]

2.3.1. HDFS

Das Hadoop Distributed File System (HDFS) ist ein verteiltes Dateisystem, das für den Einsatz auf handelsüblicher Hardware konzipiert ist. Es verfolgt einen Master/Slave-Architekturansatz. Das Hadoop Distributed File System (HDFS) besteht aus zwei Hauptkomponenten: dem Namenode und den Datanodes.[17]

Das HDFS ist auf hohe Fehlertoleranz, Streaming-Datenzugriff, Verwaltung großer Datenmengen, Einfachheit und Portabilität ausgelegt. Hardwareausfälle sind in HDFS-Instanzen mit Hunderten oder Tausenden von Servern die Norm, nicht die Ausnahme, daher ist die schnelle Fehlererkennung und -behebung ein zentrales Architekturziel. HDFS ist auf Streaming-Datenzugriff und hohen Datendurchsatz ausgerichtet, ideal für Anwendungen, die große Datenmengen verarbeiten. Ein typisches HDFS-Dateiformat reicht von Gigabyte bis Terabyte Größe, und es ist auf hohe aggregierte Datenbandbreite und Skalierbarkeit ausgelegt.[18]

---

[16] https://www.itmagazine.ch/imgserver/artikel/Illustrationen/2014/mid/ITM09_50.jpg_140908_110954.jpg

[17] Dhruba Borthakur, S. 3

[18] Dhruba Borthakur, S. 3-4

Die Einfachheit des Systems wird durch das Schreib-einmal-lese-viele Zugriffsmodell erreicht, das die Datenkohärenzprobleme vereinfacht und einen hohen Datendurchsatz ermöglicht. Eine wichtige Annahme von HDFS ist, dass das Verschieben von Berechnungen kostengünstiger ist als das Verschieben von Daten, wodurch Netzwerkstaus minimiert und der Gesamtdurchsatz des Systems erhöht wird.[19]

2.3.1.1. Namenodes und Datanodes

Namenode: Dies ist der Master-Server, der den Dateisystem-Namensraum verwaltet und den Zugriff auf Dateien durch Clients reguliert. Der Namenode führt Dateisystemoperationen wie das Öffnen, Schließen und Umbenennen von Dateien und Verzeichnissen aus. Er bestimmt auch die Zuordnung von Blöcken zu Datanodes.[20]

Datanodes: Dies sind die Slave-Server, in der Regel gibt es einen pro Knoten im Cluster. Sie verwalten den an die Knoten angeschlossenen Speicher. Intern wird eine Datei in einen oder mehrere Blöcke aufgeteilt und diese Blöcke werden in einem Satz von Datanodes gespeichert. Die Datanodes sind verantwortlich für das Bedienen von Lese- und Schreibanforderungen von den Clients des Dateisystems. Sie führen auch die Blockerstellung, -löschung und -replikation auf Anweisung des Namenodes aus.[21]

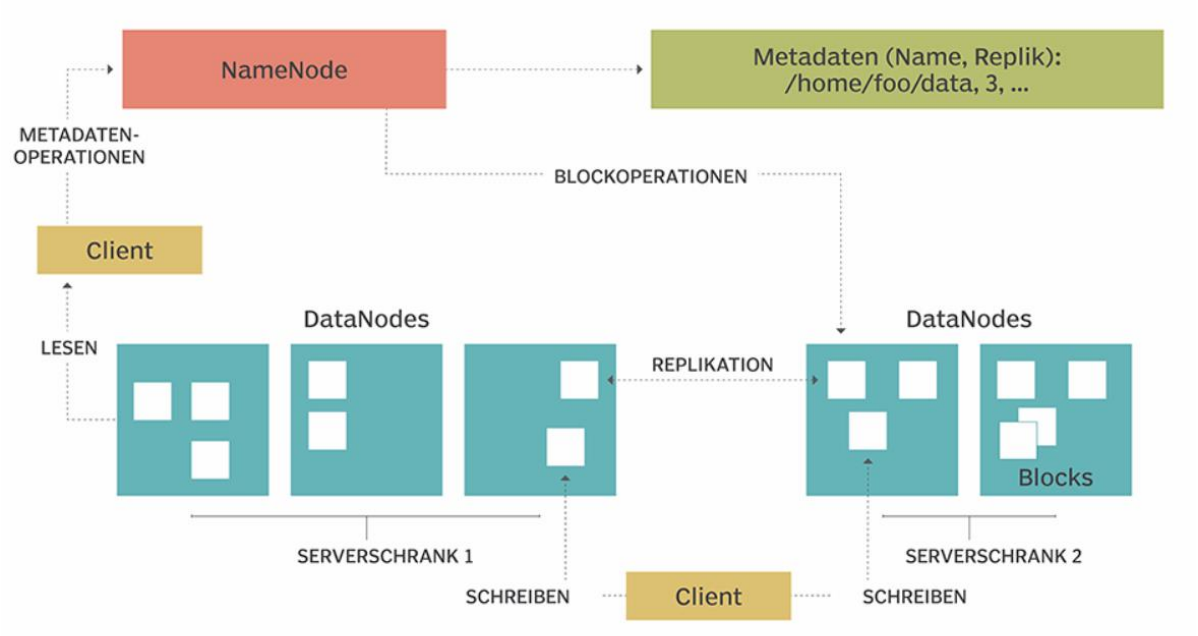


Abbildung 3 - HDFS Architektur[22]

Die Struktur von HDFS-Dateien besteht aus Blöcken. Für jeden benötigten Block vergibt der NameNode eine eindeutige Block-ID und legt eine Liste von DataNodes zur Speicherung der Blockreplikate fest. Diese Datenpakete werden in einer Pipeline sequenziell von

[19] Dhruba Borthakur, S. 4
[20] Dhruba Borthakur, S. 4-5
[21] Dhruba Borthakur, S. 4-5
[22] https://cdn.ttgtmedia.com/rms/German/Die-HDFS-Architektur-deutsch.png

DataNode zu DataNode geschrieben, wobei die Reihenfolge der DataNodes so gewählt ist, dass die Netzwerkentfernung minimiert wird.[23]

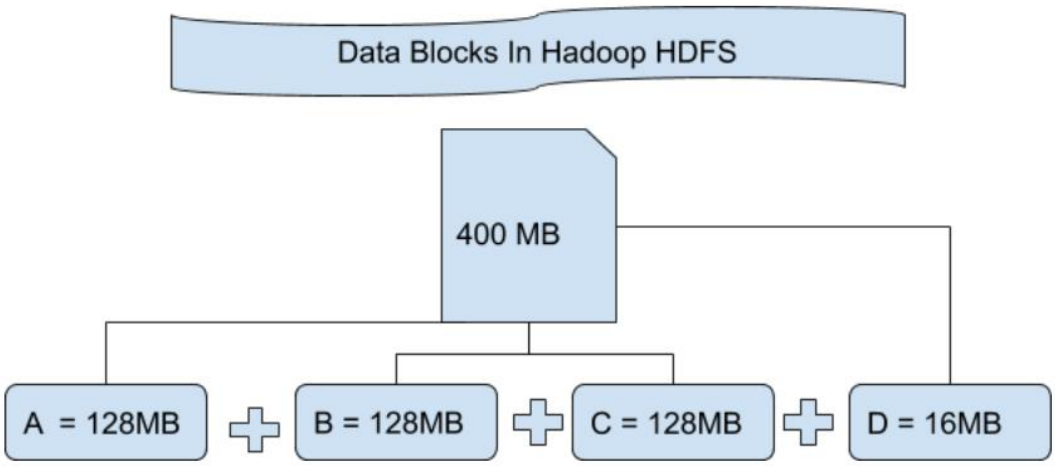


Abbildung 4 - HDFS Datei in Blöcken[24]

2.3.1.2. Lese- und Schreiboperationen

HDFS arbeitet nach einem Schreib- und Leseprinzip, das einen Schreiber und mehrere Leser vorsieht. Ein schreibender Client erhält eine periodisch erneuerbare, exklusive Zugriffsberechtigung (Lease) für die Datei, die jedoch das Lesen durch andere Clients nicht blockiert. Die Sichtbarkeit der Daten ist grundsätzlich erst nach dem Schließen der Datei garantiert.[25]

HDFS ist primär für Batch-Verarbeitungssysteme konzipiert, die einen hohen Durchsatz für sequenzielle Lese- und Schreibvorgänge benötigen. Es gibt jedoch Bemühungen, die Reaktionszeit für Lese-/Schreibvorgänge zu verbessern, um Anwendungen mit Echtzeit-Datenströmen oder zufälligem, Echtzeit-Zugriff auf große Tabellen zu unterstützen.[26]

[23] Shvachko u. a., 2010, S. 5
[24] https://media.geeksforgeeks.org/wp-content/uploads/20200621121959/3164-1.png
[25] Shvachko u. a., 2010, S.4 -5
[26] Shvachko u. a., 2010, S. 5

2.3.1.3. HDFS Client

HDFS ermöglicht Benutzeranwendungen den Zugriff auf das Dateisystem durch den HDFS-Client, eine Codebibliothek, die die HDFS-Dateisystem-Schnittstelle bereitstellt. Anders als herkömmliche Dateisysteme stellt HDFS eine API bereit, die die Positionen von Dateiblöcken anzeigt. Das verbessert die Leseleistung, da Anwendungen ähnlich dem MapReduce-Modell Aufgaben dort planen können, wo die Daten liegen.[27]

Wenn eine Datei gelesen wird, stellt der HDFS-Client zunächst eine Anfrage an den Namenode, um eine Liste der Datanodes zu erhalten, die die Repliken der Blöcke dieser Datei speichern. Anschließend nimmt der Client direkt Kontakt zu einem Datanode auf und fordert die Übertragung des benötigten Blocks.[28] :

1. Client öffnet die zu lesende Datei mittels open() auf dem Distributed File System (DFS).
2. DFS kommuniziert mit dem Namenknoten, um die Standorte der ersten Dateiblöcke zu bestimmen und gibt ein FSDataInputStream an den Client zurück.
3. Der Client ruft read() auf dem Stream auf, wobei das DFSInputStream eine Verbindung zum ersten Datenknoten herstellt.
4. Daten werden vom Datenknoten zum Client gestreamt.
5. Bei Erreichen des Blockendes schließt DFSInputStream die Verbindung und sucht den nächsten Datenknoten. Es holt auch bei Bedarf neue Datenknotenstandorte vom Namenknoten.
6. Nach dem Lesen der Datei ruft der Client close() auf dem FSDataInputStream auf.

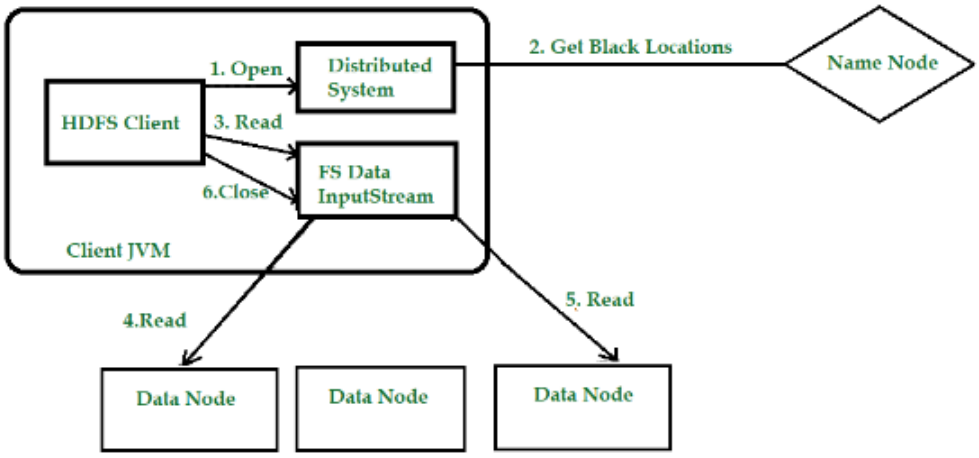


Abbildung 5 - HDFS Client Read[29]

[27] Shvachko u. a., 2010, S. 2-3
[28] GeeksForGeeks, 2023, https://www.geeksforgeeks.org/anatomy-of-file-read-and-write-in-hdfs/
[29] https://media.geeksforgeeks.org/wp-content/uploads/20200611181201/HDFS_Read.png

Im Fall eines Schreibvorgangs, initiiert der Client beim Namenode eine Anfrage zur Auswahl von Datanodes, die Repliken des ersten Blocks der Datei hosten sollen. Der Client erstellt daraufhin eine Pipeline und übermittelt die Daten. Sobald der erste Block komplett ist, werden vom Client neue Datanodes für die Aufnahme des folgenden Blocks angefragt.[30] :

1. Der Client erstellt die Datei, indem er create() auf dem DistributedFileSystem (DFS) aufruft.
2. DFS sendet einen RPC-Aufruf an den Namenknoten, um eine neue Datei zu erstellen, mit keinen zugeordneten Blöcken. Bei erfolgreicher Überprüfung erstellt der Namenknoten einen Eintrag für die neue Datei.
3. Wenn der Client Daten schreibt, teilt DFSOutputStream sie in Pakete und sendet sie an eine interne Daten-Warteschlange. Der Datenstromer fordert den Namenknoten auf, neue Blöcke zu allokieren und wählt dazu geeignete Datenknoten aus.
4. Die Datenknoten speichern die Pakete und leiten sie an den nächsten Knoten in der Pipeline weiter.
5. DFSOutputStream hält eine interne Warteschlange von Paketen, die auf eine Bestätigung von den Datenknoten warten.
6. DFSOutputStream sendet alle verbleibenden Pakete an die Datenknoten-Pipeline und wartet auf Bestätigungen, bevor es den Namenknoten informiert, ob die Datei vollständig ist oder nicht.

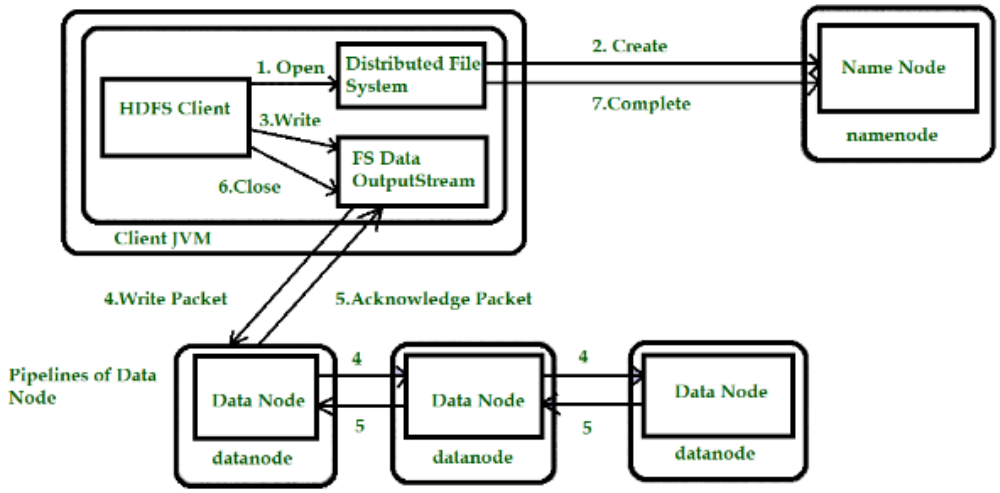


Abbildung 6 - HDFS Client Write[31]

[30] GeeksForGeeks, 2023, https://www.geeksforgeeks.org/anatomy-of-file-read-and-write-in-hdfs/
[31] https://media.geeksforgeeks.org/wp-content/uploads/20200611181133/HDFS_write-1.png

2.3.1.4. Datenreplikation

Die Platzierung von Replikaten im HDFS ist essenziell für Leistung und Zuverlässigkeit des Systems. Sie wird durch den Namenode gesteuert, welcher die Rack-Zugehörigkeit jedes Datanode bestimmt. Wenn der Replikationsfaktor drei ist, legt HDFS eine Replik auf der lokalen Maschine ab, wenn der Schreibvorgang auf einem Datanode stattfindet, oder auf einem zufälligen Datanode im selben Knoten wie der Schreibvorgang. Zwei weitere Repliken werden auf unterschiedlichen Knoten in einem anderen Rack platziert.[32]

Bei einem Replikationsfaktor größer als drei werden zusätzliche Replikate zufällig platziert, wobei die Anzahl der Replikate pro Rack unterhalb einer Obergrenze bleibt, die durch die Formel *(Replikate - 1) / Racks + 2* gegeben ist. Diese Strategie soll eine Balance zwischen Schreibperformance, Datenzuverlässigkeit und Lesegeschwindigkeit bieten.[33]

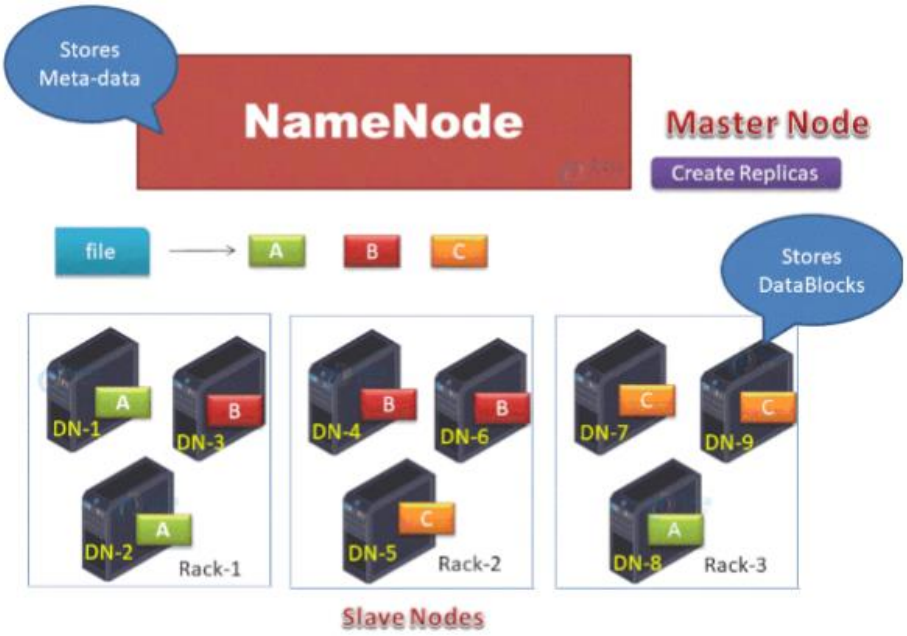


Abbildung 7 - Replikas in Datanodes[34]

[32] Apache, 2023, https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html
[33] Apache, 2023, https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html
[34] https://media.licdn.com/dms/image/C4E12AQHZUggkhbx3JQ/article-cover_image-shrink_720_1280/0/1602430399757?e=2147483647&v=beta&t=53VTI1gR-EE6W1IVX-1exEQuxZhLgoMqM_OGEXEXGlw

2.3.1.5. Datenintegrität

Bei großen Datenmengen, wie sie Hadoop verarbeiten kann, ist die Wahrscheinlichkeit von Datenbeschädigungen hoch. Zur Erkennung solcher Fehler werden Prüfsummen (Checksummen) berechnet, sobald Daten in das System eingespeist oder über unsichere Kanäle übertragen werden.[35]

HDFS erstellt automatisch eine Checksumme für alle geschriebenen Daten und prüft diese standardmäßig beim Lesen der Daten. Die Prüfsumme wird alle 512 Bytes (Standardwert) neu berechnet. Datenknoten (Datanodes) verifizieren vor der Speicherung sowohl die Daten als auch deren Prüfsummen. Bei einer erkannten Fehlübertragung erhält der Client eine IOException.[36]

Clients verifizieren ebenfalls die Checksummen beim Lesen der Daten und vergleichen sie mit den auf den Datenknoten gespeicherten Werten. Zusätzlich überprüft jeder Datenknoten in regelmäßigen Abständen alle auf ihm gespeicherten Blöcke auf Bit-Fehler[37]

2.3.1.6. Fehlermanagement

Wenn ein Client einen Fehler beim Lesen eines Blocks feststellt, meldet er den schlechten Block und den betroffenen Datenknoten an den Namenode. Da HDFS mehrere Repliken von Blöcken speichert, kann es beschädigte Blöcke "heilen", indem es eine der guten Repliken kopiert. Der Namenode überwacht kontinuierlich die Notwendigkeit zur Replikation von Blöcken und setzt diese ein, wenn es notwendig ist.[38]

Um zu prüfen ob Datanodes weiterhin erreichbar sind werden Periodische Heartbeat-Nachrichten von jedem Datanode an den Namenode gesendet. Bei fehlender Heartbeat-Nachricht erkennt der Namenode, dass eine Netzwerkunterbrechung vorliegt und einige Datanodes die Verbindung verloren haben. Diese als "tot" klassifizierten Datanodes erhalten keine weiteren IO-Anfragen. Falls der Replikationsfaktor eines Blocks dadurch unter das vorgegebene Niveau sinkt, wird das entsprechende DataNode repliziert.[39]

---

[35] White, 2015, S. 97
[36] White, 2015, S. 98
[37] White, 2015, S. 98
[38] White, 2015, S: 97-98
[39] Dhruba Borthakur, S. 10

2.3.2. YARN

Apache YARN (Yet Another Resource Negotiator) ist das Ressourcenmanagement-System von Hadoop. Es wurde in Hadoop 2 eingeführt, um die MapReduce-Implementierung zu verbessern, kann aber auch andere verteilte Berechnungsparadigmen unterstützen. Benutzer interagieren normalerweise nicht direkt mit den YARN-APIs, sondern schreiben höherstufige APIs, die von verteilten Rechen-Frameworks bereitgestellt werden. Diese Frameworks, wie MapReduce laufen als YARN-Anwendungen auf den Cluster-Compute- und Speicherebenen (HDFS und HBase[40]

In der ursprünglichen Version von Hadoop (Version 1 und früher) kontrollieren zwei Arten von Daemons den Jobausführungsprozess: ein JobTracker und mehrere TaskTracker. Im Unterschied dazu werden diese Verantwortlichkeiten in YARN von getrennten Einheiten übernommen: dem Ressourcenmanager und einem Anwendungsmaster.[41]

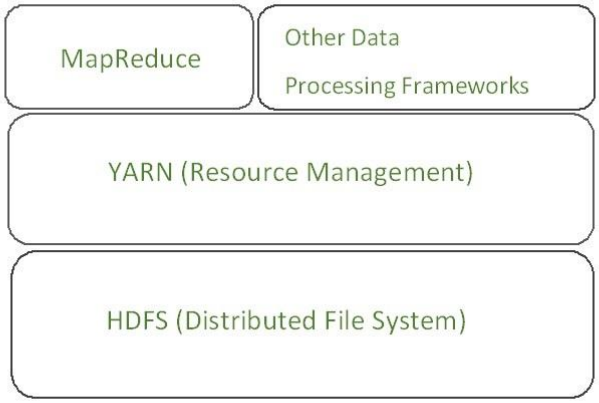


Abbildung 8 - HDFS, YARN, MapReduce - Stack[42]

YARN wurde entworfen, um viele der Einschränkungen in MapReduce 1 zu überwinden. Die Vorteile von YARN umfassen[43] :

1. Skalierbarkeit: YARN kann auf größeren Clustern als MapReduce 1 laufen. Es ist so konzipiert, dass es bis zu 10.000 Knoten und 100.000 Aufgaben skalieren kann.

2. Verfügbarkeit: Mit den Aufgaben des JobTrackers, die in YARN zwischen dem Ressourcenmanager und dem Anwendungsmaster aufgeteilt werden, wurde die Bereitstellung einer hohen Verfügbarkeit einfacher.

3. Auslastung: Im Gegensatz zu MapReduce 1, wo jeder TaskTracker mit einer statischen Zuweisung von festen "Slots" konfiguriert ist, verwaltet ein Node Manager in YARN einen Pool von Ressourcen.

---

[40] White, 2015, S. 79
[41] White, 2015, S. 79 -81
[42] https://media.geeksforgeeks.org/wp-content/uploads/Hadoop2.0-1.jpg
[43] White, 2015, S. 84-85

4. Mehrmandantenfähigkeit: YARN erweitert Hadoop um andere Arten von verteilten Anwendungen jenseits von MapReduce. Es ist sogar möglich, verschiedene Versionen von MapReduce auf demselben YARN-Cluster auszuführen.

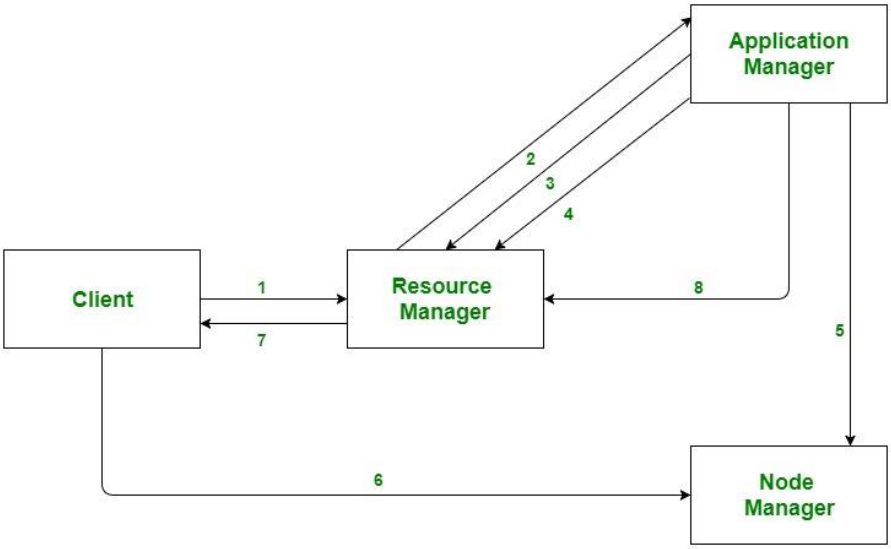


Abbildung 9 - YARN Architektur[44]

Die Hauptkomponenten der YARN-Architektur umfassen[45] :

1. **Client:** Er reicht MapReduce-Aufträge ein.
2. **Ressourcenmanager**: Dies ist die zentrale Komponente in YARN und ist für die Zuteilung und Verwaltung von Ressourcen unter allen Anwendungen verantwortlich. Er leitet Verarbeitungsanfragen an den entsprechenden Knotenmanager weiter und weist Ressourcen für deren Bearbeitung zu. Er besteht aus zwei Hauptkomponenten:
    - **Scheduler:** Er plant die Ausführung von Aufträgen basierend auf der Anwendungsanforderung und den verfügbaren Ressourcen. Er unterstützt Plugins wie den Capacity Scheduler und den Fair Scheduler.
    - **Anwendungsmanager:** Er nimmt Anwendungen entgegen und handelt den ersten Container für die Ausführung der Anwendung aus.
3. **Knotenmanager:** Dieser ist für die Verwaltung von einzelnen Knoten im Hadoop-Cluster zuständig und überwacht den Ressourcenverbrauch, führt die Protokollverwaltung durch und beendet Container auf Anweisung des Ressourcenmanagers.
4. **Anwendungsmaster:** Er ist für die Verhandlung von Ressourcen mit dem Ressourcenmanager und die Überwachung des Status und Fortschritts einer einzelnen Anwendung zuständig. Er fordert den Container vom Knotenmanager an und sendet dazu einen Container Launch Context (CLC), der alle notwendigen Informationen für die Ausführung der Anwendung enthält.

[44] https://media.geeksforgeeks.org/wp-content/uploads/Application_WorkFlow_YARN.jpg
[45] GeeksForGeeks, 2023, https://www.geeksforgeeks.org/hadoop-yarn-architecture/

5. **Container:** Dies ist eine Sammlung von physischen Ressourcen wie RAM, CPU-Kernen und Festplattenspeicher auf einem einzelnen Knoten. Die Container werden durch den Container Launch Context (CLC) aufgerufen, der Informationen wie Umgebungsvariablen, Sicherheitstokens und Abhängigkeiten enthält.

3. MapReduce-Programmiermodel

3.1. MapReduce Algorithmus

MapReduce ist eine auf dem YARN-Framework basierende Technologie, die für parallele, verteilte Verarbeitung in einem Hadoop-Cluster entwickelt wurde. Sie trägt zur hohen Verarbeitungsgeschwindigkeit von Hadoop bei, insbesondere wenn es um Big Data geht, wo serielle Verarbeitung nicht mehr ausreicht. Der MapReduce-Prozess besteht hauptsächlich aus zwei Phasen[46] :

Map: In der Map-Phase wird der Input, bestehend aus umfangreichen Daten, in einzelne Tuples (Schlüssel-Wert-Paare) zerlegt. Diese Tuples werden dann zur Reduce-Phase weitergeleitet.[47]

Reduce: In der Reduce-Phase werden die Tuples basierend auf ihren Schlüsseln kombiniert und eine bestimmte Operation, wie Sortierung oder Summierung, wird durchgeführt. Der resultierende Output ist das Endergebnis. Die spezifische Datenverarbeitung in der Reduce-Phase hängt dabei von den individuellen Anforderungen des jeweiligen Einsatzbereichs ab.[48]

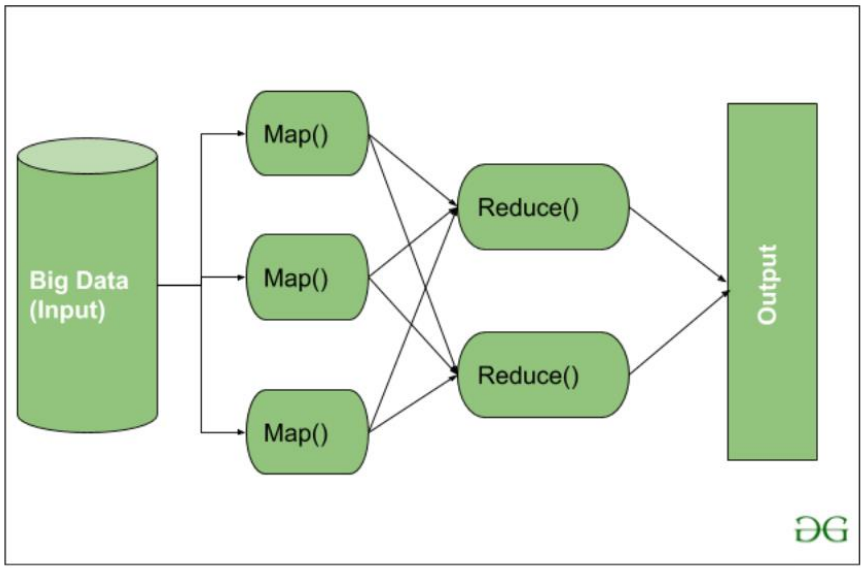


Abbildung 10 - MapReduce Workflow[49]

[46] GeeksForGeeks, 2023, https://www.geeksforgeeks.org/hadoop-architecture/
[47] GeeksForGeeks, 2023, https://www.geeksforgeeks.org/hadoop-architecture
[48] GeeksForGeeks, 2023, https://www.geeksforgeeks.org/hadoop-architecture/
[49] https://media.geeksforgeeks.org/wp-content/uploads/20200621105657/mapreduce-workflow.png

3.1. Apache MapReduce Programmiermodel

Hadoop MapReduce ist ein Software-Framework, das massenhaft Daten auf großen Hardware-Clustern verarbeitet. Es teilt Eingabedaten in unabhängige Teile auf, die parallel bearbeitet werden. Sortierte Ausgaben werden weiteren Aufgaben zugeführt, während das Framework die Aufgabenplanung und -überwachung sowie die Wiederausführung bei Fehlern übernimmt.[50]

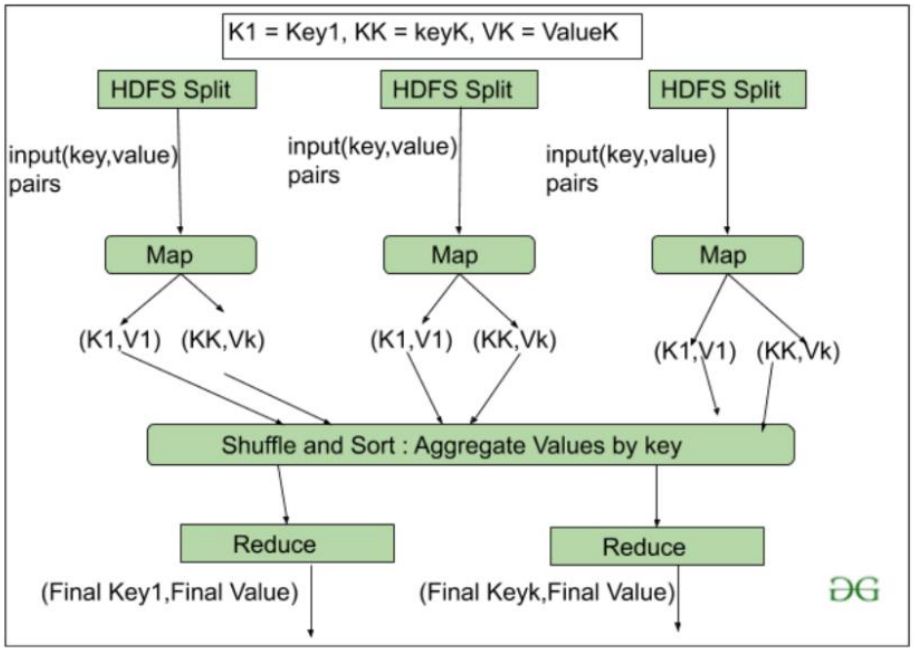


Abbildung 11 - MapReduce in Hadoop[51]

Üblicherweise werden Rechen- und Speicherknoten gemeinsam genutzt, sodass das MapReduce-Framework und das Hadoop Distributed File System auf denselben Knoten ausgeführt werden. Dies ermöglicht eine effiziente Aufgabenplanung und eine hohe Gesamtbandbreite im Cluster.[52]

Das MapReduce-Framework besteht aus einem zentralen JobTracker und einem TaskTracker pro Knoten. Der JobTracker plant und überwacht Aufgaben und führt sie bei Fehlern erneut aus, während die TaskTracker die Aufgaben ausführen. Anwendungen müssen Eingabe-/Ausgabeorte definieren und Map- und Reduce-Funktionen bereitstellen. Nach der Festlegung dieser und weiterer Auftragsparameter in der Jobkonfiguration übernimmt der Hadoop-Job-Client die Übermittlung des Jobs und der Konfiguration an den JobTracker[53]

[50] Apache, 2023, https://hadoop.apache.org/docs/r1.2.1/mapred_tutorial.html
[51] https://media.geeksforgeeks.org/wp-content/uploads/20200621115714/Map-reduce-workflow.png
[52] Apache, 2023, https://hadoop.apache.org/docs/r1.2.1/mapred_tutorial.html
[53] Apache, 2023, https://hadoop.apache.org/docs/r1.2.1/mapred_tutorial.html

3.1.1. Map Task

Die Funktionen der einzelnen Komponenten in einem MapReduce-Auftrag in Hadoop lassen sich wie folgt zusammenfassen[54]:

- R**ecordReader**: Dieses Element zerteilt Datensätze in Schlüssel-Wert-Paare, die der Map-Funktion zugeführt werden. Dabei repräsentiert der Schlüssel die Positionsinformationen und der Wert die damit assoziierten Daten.
- **Map-Funktion**: Diese benutzerdefinierte Funktion bearbeitet die von RecordReader bereitgestellten Tupel. In der Regel erzeugt die Map-Funktion entweder kein Schlüssel-Wert-Paar oder mehrere solche Paare.
- **Combiner**: Der Combiner dient dazu, Daten im Ablauf der Map-Funktion zu gruppieren, ähnlich wie ein lokaler Reducer. Er kombiniert die von der Map-Funktion erzeugten Zwischenschlüssel-Wert-Paare. Seine Verwendung ist optional.
- **Partitionierer**: Dieses Modul ist dafür verantwortlich, die in der Mapper-Phase generierten Schlüssel-Wert-Paare mit ihrem Hashcode zu sammeln. Es erstellt die Shards, die jedem Reducer zugeordnet sind.

3.1.2. Reduce Task

Nachdem der Map Task durchgeführt wurde, arbeiten die Komponenten an dem Reduce Task. Dieser wird schon während des Map Task initiiert, so bald einige Cluster mit der Map-Phase abgeschlossen haben, um die Geschwindigkeit zu erhöhen[55]:

- S**huffle and Sort**: In diesem Schritt werden die von der Map-Funktion erzeugten Zwischenschlüssel-Wert-Paare an die Reducer-Aufgabe übertragen, und über ihre Schlüsselwerte sortiert.
- **Reducer:** Die von der Map-Funktion erzeugten Tupel werden gesammelt und je nach Schlüsselelement Sortier- und Aggregationsprozesse durchzuführen.
- **Output**: Nachdem alle Operationen durchgeführt wurden, werden die Schlüssel-Wert-Paare mithilfe eines Datensatzschreibers in eine Datei geschrieben.

3.2.3 Java MapReduce Codebeispiel

In dem gegebenen Java-Codebeispiel geht es um eine MapReduce-Anwendung, die auf einem großen Wetterdatensatz ausgeführt wird. Die Hauptaufgabe besteht darin, die höchste Temperatur für jedes Jahr zu finden.

Es ist zu beachten, dass Hadoop seine eigenen grundlegenden Datentypen bereitstellt, die für die Netzwerkserialisierung optimiert sind. In diesem Codebeispiel verwenden wir **LongWritable** (entspricht einem Java Long), **Text** (wie Java String) und **IntWritable** (wie Java Integer).

[54] GeeksForGeeks, 2023, https://www.geeksforgeeks.org/hadoop-architecture/
[55] GeeksForGeeks, 2023, https://www.geeksforgeeks.org/hadoop-architecture/

Der Prozess läuft in drei Hauptteilen ab:

```
1  // Importieren der benötigten Bibliotheken
2  import java.io.IOException;
3  import org.apache.hadoop.io.IntWritable;
4  import org.apache.hadoop.io.LongWritable;
5  import org.apache.hadoop.io.Text;
6  import org.apache.hadoop.mapreduce.Mapper;
7  import org.apache.hadoop.mapreduce.Reducer;
8  import org.apache.hadoop.fs.Path;
9  import org.apache.hadoop.mapreduce.Job;
10 import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
11 import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
```

Listing 1 - MaxTemperatur MapReduce Codebeispiel - Imports[56]

Mapper: Der Mapper nimmt Rohdaten des NCDC (National Climatic Data Center) als Eingabe. Die einzige Information, die für diese Aufgabe von Interesse ist, sind das Jahr und die Lufttemperatur. Der Mapper bereitet die Daten so auf, dass die Reduce-Funktion sie weiterverarbeiten kann. Er filtert außerdem ungültige, fehlende oder fehlerhafte Temperaturdaten aus. Der Code für den Mapper ist in der Klasse **MaxTemperatureMapper** definiert.

```
12
13 // Definition des Mappers
14 public class MaxTemperatureMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
15     private static final int MISSING = 9999;
16
17     @Override
18     public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
19
20         String line = value.toString();
21         String year = line.substring(15, 19);
22         int airTemperature;
23
24         // Verarbeitung von Pluszeichen, da parseInt Probleme damit hat
25         if (line.charAt(87) == '+') {
26             airTemperature = Integer.parseInt(line.substring(88, 92));
27         } else {
28             airTemperature = Integer.parseInt(line.substring(87, 92));
29         }
30
31         String quality = line.substring(92, 93);
32
33         // Nur Werte schreiben, wenn die Temperatur vorhanden ist und der
34         // Qualitätswert den Temperaturwert als korrekt kennzeichnet
35         if (airTemperature != MISSING && quality.matches("[01459]")) {
36             context.write(new Text(year), new IntWritable(airTemperature));
37         }
38     }
39 }
```

Listing 2 - MaxTemperatur MapReduce Codebeispiel - Mapper[57]

Reducer: Die Hauptaufgabe des Reducers ist es, die von der Map-Funktion generierten Tupel zu sammeln und die maximale Temperatur für jedes Jahr zu finden. Der Code für den Reducer ist in der Klasse **MaxTemperatureReducer** definiert.

[56] White, 2015, S. 153
[57] White, 2015, S. 153

```java
// Definition des Reducers
public class MaxTemperatureReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    @Override
    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException,
    InterruptedException {

        int maxValue = Integer.MIN_VALUE;

        // Durchlaufen aller Werte und Finden des maximalen Werts
        for (IntWritable value : values) {
            maxValue = Math.max(maxValue, value.get());
        }
        context.write(key, new IntWritable(maxValue));
    }
}
```

Listing 3 - MaxTemperatur MapReduce Codebeispiel - Reducer[58]

JobRunner: Dieser Teil des Codes ist verantwortlich für das Ausführen des MapReduce-Jobs. Es spezifiziert die Eingabe- und Ausgabepfade, die Mapper- und Reducer-Klassen sowie die Ausgabe-Schlüssel- und Wertklassen. Der Code dafür ist in der Klasse **MaxTemperature** definiert.

```java
// Hauptklasse, die den MapReduce-Job ausführt
public class MaxTemperature {

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: MaxTemperature <input path> <output path>");
            System.exit(-1);
        }

        Job job = new Job();
        job.setJarByClass(MaxTemperature.class);
        job.setJobName("Max temperature");

        // Setzen der Eingabe- und Ausgabepfade
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        // Setzen der Mapper- und Reducer-Klassen
        job.setMapperClass(MaxTemperatureMapper.class);
        job.setReducerClass(MaxTemperatureReducer.class);

        // Setzen der Ausgabe-Schlüssel- und Wertklassen
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        // Abschicken des Jobs und Warten auf dessen Beendigung
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Listing 4 - MaxTemperatur MapReduce Codebeispiel - JobRunner[59]

[58] White, 2015, S. 154
[59] White, 2015, S. 154-155

4. Hadoop und MapReduce in der Praxis

4.1. Anwendungsbeispiele

4.1.1 Netflix

Netflix nutzt Hadoop in großem Maßstab in seiner Cloud-basierten Datenbankarchitektur, die es ihnen ermöglicht, ein Datenlager von praktisch unendlichem Umfang zu erstellen. Sie nutzen die Flexibilität der Cloud, um ein dynamisch skalierbares System zu erstellen.

Netflix hat eine eigene Hadoop-Plattform als Service (PaaS) namens "Genie" entwickelt, die REST-APIs für Jobausführung und Ressourcenverwaltung bietet. Genie ermöglicht es Benutzern, einzelne Hadoop-, Hive- und Pig-Jobs über eine REST-API zu übermitteln, ohne neue Hadoop-Cluster bereitstellen oder Hadoop-, Hive- oder Pig-Clients installieren zu müssen. Darüber hinaus ermöglicht es Administratoren, die Konfigurationen verschiedener Hadoop-Ressourcen in der Cloud zu verwalten und zu abstrahieren.

Die Entwicklung von Genie war notwendig, da es keine bestehende Lösung in der Open-Source-Gemeinschaft gab, die ihre Anforderungen erfüllte: eine API für die Jobausführung, die Abstraktion von Backend-Clustern, die Möglichkeit, Jobs an mehrere Cluster zu senden, und die Skalierbarkeit zur Unterstützung ihres Nutzungsgrades. Andere Lösungen wie Oozie oder Templeton wurden in Betracht gezogen, erfüllten jedoch nicht alle Anforderungen von Netflix, beispielsweise die Unterstützung von Hive oder die Möglichkeit, gleichzeitig Jobs an mehr als einen Cluster zu senden.[60]

4.2.2 Facebook

Facebook Messages ist die erste benutzerorientierte Anwendung von Facebook, die auf der Apache Hadoop-Plattform basiert. Apache HBase ist eine datenbankähnliche Schicht, die auf Hadoop aufgebaut ist und dazu ausgelegt ist, Milliarden von Nachrichten pro Tag zu unterstützen.

Facebook wählte Hadoop und HBase über andere Systeme wie Apache Cassandra und Voldemort aufgrund der Anforderungen der Anwendung an Konsistenz, Verfügbarkeit, Partitionstoleranz, Datenmodell und Skalierbarkeit. Facebook hat Verbesserungen an Hadoop vorgenommen, um es zu einem effektiveren Echtzeitsystem zu machen.

Dieser Ansatz bietet bedeutende Vorteile gegenüber dem sharded MySQL-Datenbankschema, das in anderen Anwendungen bei Facebook und vielen anderen Web-Scale-Unternehmen verwendet wird.[61]

---

[60] Blog, 2013, https://netflixtechblog.com/hadoop-platform-as-a-service-in-the-cloud-c23f35f965e7
[61] Sellis, 2011, S. 1

4.2.3 Twitter

Twitter verwendet Apache Hadoop und hat aufgrund des Fehlens von standardisierten Cluster-Management-Tools den Data Platform Cluster Operator (DCO) Service entwickelt. Dieser in Python erstellte Service zielt darauf ab, die Betriebseffizienz zu erhöhen und robuste Lösungen für das Management von Hadoop-Clustern bereitzustellen.

Vor der Einführung des DCO mussten Site Reliability Engineers (SREs) im Data Platform Team einen Großteil ihrer Zeit für das Management und die Wartung von Hadoop-Cluster-Operationen aufwenden. Dies umfasste das Hinzufügen oder Entfernen von Hosts zu Clustern, das Erfüllen von Kapazitätsanforderungen der Benutzer, das Leeren von Hosts vor der Entfernung, das Aufrechterhalten des Lebenszyklus der Hosts und das Gewährleisten der Cluster-Gesundheit.

Der DCO-Service hat diese Prozesse optimiert und das Cluster-Management zu einem nahtlosen Erlebnis gemacht. Zusammenfassend bringt der DCO-Service erhebliche Fortschritte in Bezug auf Betrieb, Wartung und Skalierbarkeit für das SRE-Team. Der Service hat umfassende Tests durchlaufen und wurde erfolgreich auf Produktionsclustern eingesetzt, wodurch die Infrastruktur robuster und zuverlässiger wurde.[62]

4.2. Limitationen von Hadoop MapReduce

Fallstudie: Spark: Cluster computing with working sets. HotCloud

Dieses Paper stellt ein neues Cluster-Computing-Framework namens Spark vor, dass Anwendungen mit Arbeitsdatensätzen unterstützt und dabei ähnliche Skalierbarkeit und Fehlertoleranz wie MapReduce bietet. Die Hauptabstraktion in Spark sind resiliente verteilte Datensätze (Resilient Distributed Datasets, RDDs), die eine schreibgeschützte Sammlung von Objekten darstellen, die über einen Satz von Maschinen verteilt sind und wieder aufgebaut werden können, falls eine Partition verloren geht.

Die Autoren des Papers heben zwei Anwendungsfälle hervor, in denen MapReduce nicht effizient ist: iterative Jobs und interaktive Analysen. Iterative Jobs, wie viele maschinelle Lernalgorithmen, wenden eine Funktion wiederholt auf denselben Datensatz an, um einen Parameter zu optimieren. Mit MapReduce muss jeder Job die Daten neu von der Festplatte laden, was zu erheblichen Performanceeinbußen führt. Bei interaktiven Analysen, wie explorativen Abfragen auf großen Datensätzen, führt jeder Abfragevorgang mit Hadoop zu erheblicher Latenz, da jeder Job als separate MapReduce-Aufgabe ausgeführt wird und Daten von der Festplatte liest.[63]

[62] Twitter, 2023, https://blog.twitter.com/engineering/en_us/topics/infrastructure/2023/the-data-platform-cluster-operator-service-for-hadoop-cluster-management
[63] Zaharia, 2013, S.1, 5-6

5. Übung: MapReduce-Aufgabe

5.1. Erläuterung der Aufgabenstellung

1. Erstellen sie eine Map-Funktion und eine Reduce-Funktion mit dessen Hilfe ein Job in Apache Hadoop die Anzahl von einem Token/Wort aus einer Textdatei lesen kann.

Für die Map-Funktion gibt es die Klasse StringTokenizer die vom MapReduce Framework bereitgestellt wird:

new StringTokenizer(String input) -> Erstellt einen Tokenizer für ein String

StringTokenizer.hasMoreTokes() -> Es existieren weitere Tokens/Wörter im String

StringTokenizer.nextToken() -> gibt den nächsten Token zurück und setzt den Iterator auf den nächsten Token

In diesem Fall müssen sie statt context.write() den Befehl output.collect(Text key, IntWritable value) verwenden.

2. Visualisieren Sie nun die Schritte die ihr MapReduce Job für Sie erledigt. Sie können dafür 2 Cluster als Ausganspunkt nehmen die Daten einer Textdatei beinhalten. Wenn sie das Beispiel ganz einfach halten wollen, können Sie als String einfach ein Set aus Buchstaben verwenden.

Schritte die es zu visualisieren gilt:

- Mapping Phase
- Sort und Shuffle Phase
- Reduce Phase

5.2. Lösung zur Aufgabenstellung

1. Übungsaufgabe

Mapping-Funktion

```
private final static IntWritable one = new IntWritable(1);
private Text word = new Text();

public void map(LongWritable key, Text value, OutputCollector<Text, IntWritable> output, Reporter reporter) throws IOException {
    String line = value.toString();
    StringTokenizer tokenizer = new StringTokenizer(line);

    while (tokenizer.hasMoreTokens()) {
        word.set(tokenizer.nextToken());
        output.collect(word, one);
    }

}
```

Listing 5 – Lösung Map-Funktion

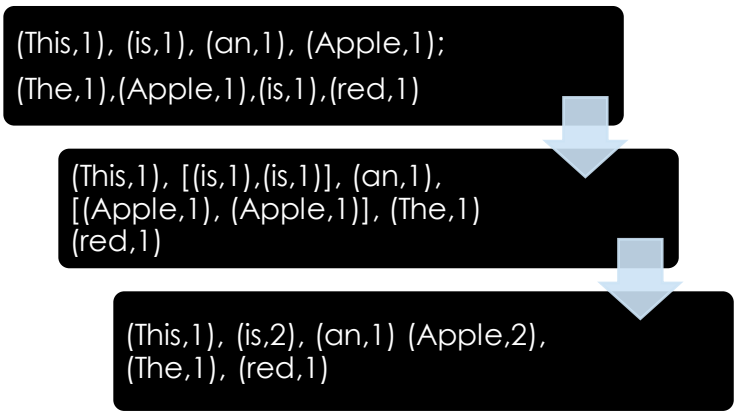
Reduce-Funktion

```
public void reduce(Text key, Iterator<IntWritable> values, OutputCollector<Text, IntWritable> output, Reporter reporter)
throws IOException {
        int sum = 0;
        while (values.hasNext()) {
         sum += values.next().get();
        }
        output.collect(key, new IntWritable(sum));
}
```

Listing 6 – Lösung Reduce-Funktion

2. Übungsaufgabe

Input: This is an Apple; The Apple is red



I. Literatur

Blog, Netflix Technology (2013): Hadoop Platform as a Service in the Cloud - Netflix Tech-Blog, in: Netflix TechBlog, v. 10.01.2013, https://netflixtechblog.com/hadoop-platform-as-a-service-in-the-cloud-c23f35f965e7, 05.06.2023

Dhruba Borthakur: The Hadoop Distributed File System: Architecture and Design, in: , 2007

Jeffrey Dean and Sanjay Ghemawat (2004): MapReduce: Simplified Data Processing on Large Clusters, in: , 2004

Sellis, Timos (2011): Proceedings of the 2011 ACM SIGMOD International Conference on Management of data, ACM Conferences, New York, NY, 2011, http://dl.acm.org/citation.cfm?id=1989323

Shvachko, Konstantin u. a. (2010): The Hadoop Distributed File System, in: , 2010, 2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST), 2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST), Incline Village, NV, USA, 03.05.2010 - 07.05.2010, S. 1–10

White, Tom (2015): Hadoop: The definitive guide, Fourth edition, Beijing, 2015

Zaharia (2013): Sparks, in: , 2013

Apache (14.04.2023): Apache Hadoop, https://hadoop.apache.org/, 03.06.2023

GeeksForGeeks (04.06.2023): Hadoop Architecture, https://www.geeksforgeeks.org/hadoop-architecture/, 04.06.2023

Twitter (05.06.2023): The data platform cluster operator service for Hadoop cluster management, https://blog.twitter.com/engineering/en_us/topics/infrastructure/2023/the-data-platform-cluster-operator-service-for-hadoop-cluster-management, 05.06.2023